UNIVERSITÀ CATTOLICA DEL SACRO CUORE

Sede di Milano

Facoltà di Economia

Corso di Laurea in Economia e Gestione Aziendale

MapReduce: un paradigma di computazione per big data

Relatore: Prof. Jianyi Lin

Tesi di laurea di:

Alberto Venturini

N. Matr. : 4903185

Anno Accademico 2021/2022

Indice

Introduzione *3*

1. Introduzione a MapReduce *4*

1.1 Definizione e cenni storici *4*

1.2 L'architettura del modello MapReduce *5*

1.3 Le fasi chiave della computazione *6*

1.4 Vantaggi e limiti del modello *7*

1.5 Comparazione con altre tecnologie per elaborare Big Data *8*

2. Il problema del conteggio delle parole in MapReduce *10*

2.1 Conteggio delle parole in un testo *10*

2.2 Implementazione con MapReduce *11*

3. L'algoritmo PageRank in MapReduce *15*

3.1 **i primi motori di ricerca** *15*

3.2 **il modello PageRank** *17*

3.3 **Limiti e svantaggi nella formulazione base dell'algoritmo PageRank** *20*

3.4 **Ottimizzazione dell'implementazione di MapReduce nell'algoritmo PageRank** *24*

3.5 **Computazione effettiva dell'algoritmo PageRank su un grafo reale** *26*

Conclusioni *31*

Bibliografia *32*

Introduzione

I Big Data sono ormai diventati una componente critica nell'era della rivoluzione digitale, grazie ad un aumento esponenziale del flusso di dati che riusciamo a raccogliere, immagazzinare ed elaborare [2]. I big data provengo da numerosi fonti come i social-media, gli e-commerce, i processi delle scienze naturali, i fenomeni fisici, ecc. Le tecniche tradizionali per elaborare ed analizzare i dati risultano insufficienti di fronte alla grandezza e alla complessità dei Big Data, caratterizzati da: Volume, Velocità, Varietà, Veridicità e Valore. Per questo motivo la ricerca di nuove tecniche di analisi e nuovi paradigmi di computazione rimane un problema di attuale interesse al fine di facilitare e rendere più efficiente l'intero ciclo di gestione di tali dati.

Il modello MapReduce è stato introdotto da Google nel 2004 [1] e da allora è diventato uno strumento molto utilizzato per elaborare dati in larga scala. L'obiettivo di questa tesi è esplorare il modello di programmazione MapReduce e le sue diverse applicazioni per dimostrarne l'efficacia nel risolvere problemi di analisi legati ai Big Data in maniera efficiente attraverso l'implementazione di esempi pratici.

Inizialmente la tesi introduce il background storico del modello MapReduce, ed in seguito elabora un esempio di conteggio di parole utilizzando il modello MapReduce esponendo i relativi vantaggi e svantaggi [1].
Parleremo poi di una delle più famose implementazioni del modello MapReduce: l'algoritmo PageRank, un algoritmo molto conosciuto a livello industriale per ordinare le pagine Web rappresentate in un grafo.

L'implementazione di PageRank richiede alcuni accorgimenti tecnici e per questo vedremo come la sua realizzazione può essere ottimizzate attraverso l'utilizzo di MapReduce [1]. Infine, evidenzieremo i vantaggi principali derivanti dall'adozione modello MapReduce e quali sono le direzioni future per la ricerca su questo modello. [3]

1. Introduzione al MapReduce

Questo capitolo fornirà un'introduzione a MapReduce, un noto modello di programmazione per svolgere determinate elaborazioni di grandi dati. Copriremo la definizione base di MapReduce ed un background storico sul modello; inoltre, si parlerà delle sue componenti chiave quale la fase di mappatura, la fase di 'shuffle' e la fase di riduzione [1][2]. Il capitolo tratterà i vantaggi e i punti deboli di MapReduce, tra cui la sua facilità d'uso, la scalabilità e la tolleranza ai guasti, nonché i suoi limiti in termini di complessità e prestazioni [1]. Inoltre, confronteremo MapReduce con altre tecnologie di elaborazione dei big data, come Hadoop, Spark e Flink, per fornire una comprensione completa dei suoi punti di forza e di debolezza [1][2].

1.1 Definizione e cenni storici di MapReduce

La storia di MapReduce risale ai primi anni 2000, quando la crescita di internet e la crescente disponibilità di big data hanno creato la necessità di una soluzione più efficiente e scalabile per l'elaborazione e la processazione di grandi set di dati. In quel periodo l'elaborazione di grandi collezioni di dati era un compito difficoltoso e dispendioso sia in termini di tempo sia in termini di calcolo e ciò richiedeva hardware e software molto specializzati e costosi.

Nel 2003, i ricercatori di Google Jeffrey Dean e Sanjay Ghemawat proposero l'idea di MapReduce come un modo per elaborare grandi set di dati che non potevano essere immagazzinati nella memoria di una singola macchina.

L'idea alla base di MapReduce era quella di dividere l'elaborazione di grandi set di dati in attività più piccole e parallele che potevano essere eseguite su più nodi in un cluster.

Il modello di programmazione MapReduce è stato introdotto per la prima volta da Google nel 2004[2].Il modello consiste in due operazioni principali: l'operazione Map e l'operazione Reduce. L'operazione Map prende un insieme di dati di input e li converte in un insieme di coppie chiave-valore intermedie, mentre l'operazione Reduce prende le coppie chiave-valore intermedie prodotte dall'operazione Map e le aggrega in un insieme di valori di output.

Implementazioni del modello MapReduce consentono di effettuare molti dei calcoli più comuni su grandi set di dati in maniera tale da tollerare fallimenti dell'hardware durante la computazione [1].

1.2 L’architettura del modello MapReduce

Spiegheremo ora cosa è un file system distribuito (‘DFS’). La maggior parte della computazione viene effettuata su una singola macchina con i suoi processori, viene anche chiamata ‘nodo di calcolo. Grazie allo sviluppo tecnologico oggi giorno i nodi di calcolo sono hardware presenti nella vita quotidiana come i personal computer, ciò riduce molto i costi rispetto agli hardware specifici per la programmazione in parallelo.

La nuova architettura per la programmazione in parallelo, chiamata anche ‘computazione in cluster’, è organizzata nella maniera seguente. I nodi di calcolo vengono posizionati su dei ‘Racks’ ovvero degli scaffali; ci sono tipicamente dagli 8 ai 64 nodi su ogni scaffale. I nodi all'interno dello scaffale sono connessi da una rete, tipicamente una rete ethernet da un gigabit. Ci possono essere più scaffali di nodi e gli scaffali sono connessi da un altro livello di rete o da uno ‘switch’ un interruttore. La larghezza della banda larga che connette gli scaffali è maggiore della rete ethernet inter-scaffale, dato il numero di nodi presenti che devono comunicare la larghezza della banda larga è essenziale. [3]

La figura 1.1 indica una possibile architettura, tuttavia possono essere presenti molti più nodi e scaffali.

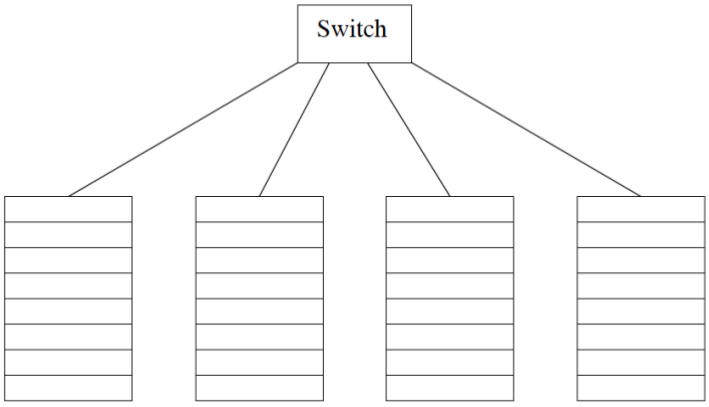

Figura 1.1 : organizzazione dei nodi negli scaffali

È risaputo che gli hardware possono rompersi e smettere di funzionare, più hardware e connessioni un sistema ha, più è frequente che qualcosa non funzioni in un momento. Alcuni calcoli richiedono minuti o addirittura ore su migliaia di nodi di calcolo. Se dovessimo abortire e ricominciare il calcolo ogni qual volta che un componente non funziona, probabilmente il calcolo non verrà mai effettuato con successo. La soluzione a questo problema prende due forme:

1. I file devono essere memorizzati in maniera ridondante. Se non duplicassimo il file in diversi nodi di calcolo, in caso di guasto di un nodo tutti i suoi file non sarebbero disponibili finché esso non verrà ripristinato. Se non avessimo eseguito un backup dei file e avvenisse un crash del disco i file andrebbero persi per sempre.
2. I calcoli devono essere divisi in compiti, in modo tale che, se un'attività fallisce ad arrivare a completamento può essere riavviata senza influenzare altre attività.[3]

1.3 Le fasi chiave del modello MapReduce

Lo stretto essenziale da scrivere per il programmatore sono due funzioni, chiamate Map e Reduce. Il sistema gestisce l'esecuzione parallela e coordina le attività che eseguono o la funzione Map o Reduce, Inoltre il sistema gestisce autonomamente il fallimento di una di queste operazioni. Possiamo quindi riassumere la computazione MapReduce nella seguente maniera:
Un file system distribuito distribuisce uno o più pezzi a un certo numero di attività Map. Il blocco diventa un elenco di coppie di valori chiave come risultato di queste attività Map. Il codice scritto dall'utente per la funzione Map controlla come vengono generate le coppie chiave-valore dai dati di input.
Un 'controller master' raccoglie e organizza le coppie chiave-valore da ogni lavoro Map in ordine chiave-valore. Tutti i lavori Reduce ricevono una parte delle chiavi, assicurando che tutte le coppie di valori chiave con la stessa chiave finiscano allo stesso compito Reduce.
L'attività Reduce lavora su una chiave alla volta, e combina tutti i valori associati a quella chiave in qualche modo. Il modo di combinazione dei valori è determinato dal codice scritto dall'utente per la funzione Reduce.
La figura 1.2 mostra come avviene questa computazione[3]

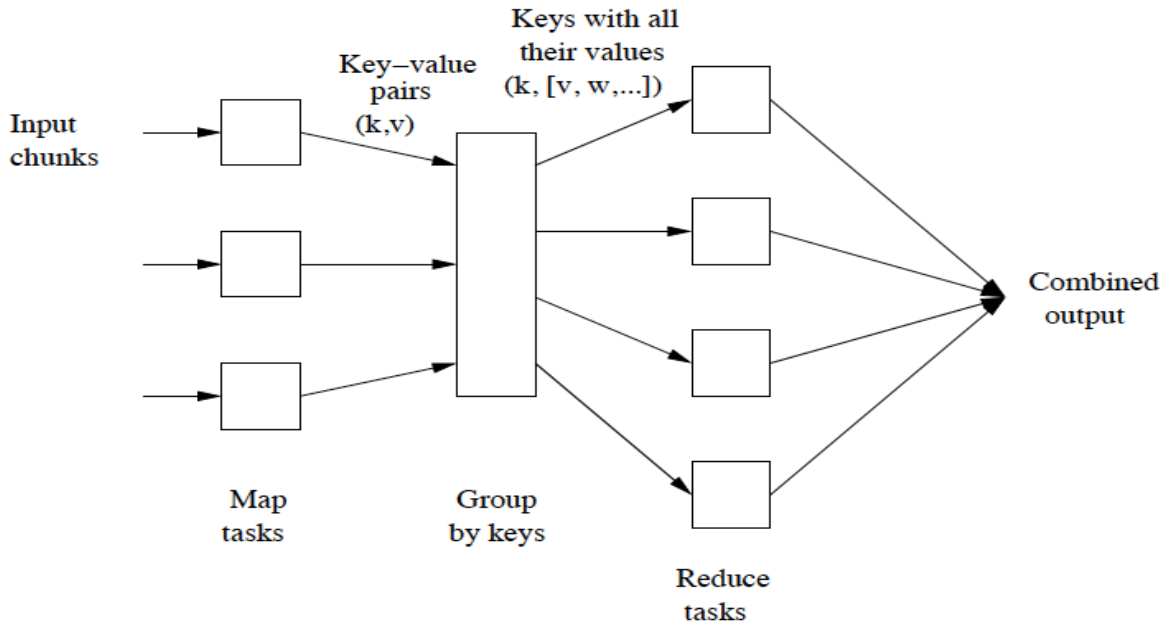

Figura 1.2 : architettura del modello MapReduce

1.4 Vantaggi e limiti del modello

MapReduce è un popolare modello di calcolo distribuito ampiamente utilizzato per l'elaborazione di big data. Ha diversi vantaggi che lo rendono ideale per la gestione di attività di elaborazione dati su larga scala.

Uno dei maggiori vantaggi di MapReduce è la sua scalabilità[4]. MapReduce può scalare orizzontalmente, il che significa che può gestire grandi quantità di dati distribuendo l'elaborazione su più nodi in un cluster. Ciò lo rende ideale per l'elaborazione di carichi di lavoro di Big Data che sarebbero difficili o impossibili da elaborare su una singola macchina.

Un altro vantaggio di MapReduce è la sua tolleranza ai guasti [1]. MapReduce è progettato per essere 'fault-tolerant', il che significa che se un nodo fallisce, l'elaborazione può essere riassegnata automaticamente a un altro nodo senza perdere alcun dato o dover riavviare l'intero processo. Ciò garantisce che l'elaborazione continui senza interruzioni, anche in caso di guasto del nodo.

MapReduce è anche un modello flessibile che può essere utilizzato per una varietà di attività di elaborazione dati, da semplici aggregazioni di dati a più complessi algoritmi di apprendimento automatico. Fornisce un modello di programmazione che consente agli sviluppatori di scrivere codice che può essere facilmente parallelizzato su un grande cluster di nodi.

Tuttavia, ci sono anche alcune limitazioni al modello MapReduce. Una delle principali limitazioni è il movimento dei dati. MapReduce richiede una quantità significativa di trasferimento dei dati tra i nodi, il che può portare a congestioni di rete causati da colli di bottiglia e a tempi di elaborazione più lunghi[4]. Ciò è particolarmente vero per i lavori MapReduce che richiedono molti trasferimenti di dati/risultati da elaborazioni intermedie tra nodi.

Un'altra limitazione di MapReduce è la sua complessità di programmazione[4]. La programmazione MapReduce può essere complessa e richiede una quantità significativa di codice per implementare attività di elaborazione dati. Questo può rendere difficile agli sviluppatori che non hanno familiarità con i modelli di calcolo distribuito iniziare con MapReduce.

1.5 Comparazione con altre tecnologie per elaborare Big Data

Recentemente, sono apparse molte tecnologie per l'elaborazione dei Big Data ciascuna con i suoi vantaggi e svantaggi. Una delle più popolari è Hadoop che comprende il Hadoop Distributed File System (HDFS) e il modello di programmazione MapReduce[4]. Hadoop è ampiamente utilizzato per l'elaborazione in lotti (batch processing) e grazie alla sua capacità di immagazzinare e trattare un quantitativo di dati molto elevato è diventato una scelta molto popolare per chi deve processare Big Data.

Tuttavia, con la crescente esigenza di elaborazione dei dati in tempo reale, il modello di elaborazione in lotti di Hadoop è diventato limitante. Ciò ha portato all'elaborazione di nuove tecnologie come Apache Spark e Apache Flink[5], le quali sono state disegnate in maniera tale da gestire sia l'elaborazione in lotti sia elaborare dati in tempo reale. Spark utilizza un modello di elaborazione dei dati in memoria che lo rende più veloce dell'elaborazione con MapReduce di Hadoop. Il calcolo in memoria in Spark si riferisce alla capacità di memorizzare e manipolare i dati nella memoria piuttosto che sul disco, consentendo un'elaborazione più veloce di grandi quantità di dati. In altre parole, Spark carica i dati in memoria e li mantiene per un accesso più rapido, invece di leggere e scrivere i dati da e su disco.

Flink utilizza un modello basato sullo streaming che offre prestazioni migliori per l'elaborazione continua dei dati. Stream processing o elaborazione dei flussi significa che l'input potrebbe non finire mai, e quindi sei costretto a elaborare continuamente i dati man mano che arrivano.[6]

Per quanto riguarda la scalabilità tutte e tre le tecnologie sono state progettate per essere scalabili orizzontalmente ovvero che possono essere facilmente scalate aggiungendo nuovi nodi al cluster. Tuttavia, Spark e Flink hanno delle migliori performance riguardanti la scalabilità grazie alla loro capacità di elaborazione distribuita, qualificandoli come i più adatti per l'elaborazione di Big Data.

Mentre Hadoop è ancora ampiamente usato, Spark e Flink stanno diventando sempre più popolari per la loro capacità di gestire sia l'elaborazione in lotti che in tempo reale. Ogni tecnologia ha i suoi punti di forza e di debolezza e la scelta di quella giusta dipende dal caso d'uso specifico e dai requisiti.[5][4]Confrontando i vantaggi e i limiti di ogni tecnologia, è possibile informarsi e prendere una decisione per scegliere la migliore tecnologia per il caso d'uso specifico.

2. Il problema del conteggio delle parole in MapReduce

Il capitolo introduce un caso pratico dove applicare il modello MapReduce, si analizzerà il problema del conteggio delle parole in un testo che viene anche considerato l'esempio base per affrontare il modello MapReduce. Dopo aver enunciato il problema e le sue varie estensioni partiche sul web andremo a vedere l'implementazione di MapReduce. In questo contesto le funzioni Map e Reduce risultano facilmente comprensibili.

2.1 Conteggio delle parole in un testo

Nel contesto dei Big Data è possibile imbattersi nella sfida di categorizzare dei documenti in base a diversi fattori come, per esempio, di che argomento trattano. Per categorizzare un documento in base all'argomento che tratta potremmo cercare e identificare parole speciali riguardanti quel distinto argomento. È facile immaginare come in un articolo di golf si abbia una alta presenza delle parole: 'mazza', 'caddie' 'par', 'birdie' ecc. Una volta classificati e categorizzati dei documenti appartenenti alla categoria del golf è facile capire che le parole chiave sono parole che ricorrono in maniera insolitamente abituale; tuttavia, finché non effettuiamo la classificazione non è possibile identificare le parole precedenti come chiave.

Una tipica classificazione inizia osservando i documenti e identificando le parole chiave in quei documenti. Ragionando una prima intuizione può essere quella di collegare le parole chiave alle parole più frequenti in quei documenti, ma questa intuizione risulta essere completamente errata poiché le parole più frequenti in una serie di documenti saranno sicuramente una categoria di parole chiamata 'stop-words' alla quale appartengono gli articoli, le preposizioni, le congiunzioni o gli aggettivi. Questa categoria di parole aiuta a costruire il testo ma non trasmette importanza, quindi, vengono usualmente eliminate prima di iniziare qualsiasi tipo di classificazione. [3]

Le parole chiave dell'argomento sono tipicamente parole rare, anche se non tutte le parole rare hanno la stessa importanza. La differenza tra parole importanti e parole senza significato può essere trovata facilmente analizzando la *frequenza* di tali parole in una parzialmente breve serie di documenti.

Formalmente tale concentrazione in una parzialmente breve serie di documenti viene misurata grazie all'indice 'TF.IDF' ('Term Frequency-Inverse Document Frequency'). [3]

2.2 Implementazione con MapReduce

Per implementare il modello MapReduce nel nostro problema del conteggio delle parole è fondamentale capire come funzionano le funzioni Map e Reduce.

Alla funzione Map verrà assegnato un documento CSV oppure semplicemente un file TXT con le parole al suo interno separate da una virgola, Ora la funzione Map riceve questo input e deve generare per ogni parola la coppia di chiave-valore intermedia (parola, 1) per ogni parola presente nel testo.

In seguito, la funzione Shuffle assegna allo stesso lavoratore coppie di chiavi intermedie con la stessa chiave.

Infine, la funzione Reduce avrà come input molteplici coppie di chiave-valore e il suo compito sarà quello di produrre un dizionario dove vengono raggruppate le stesse chiavi e come valore si somma tutte le volte che quella singola chiave specifica è stata contata.

Nella sezione seguente, esamineremo l'implementazione del modello MapReduce in un problema di conteggio delle parole utilizzando Python 3 su Google Colab. Verranno forniti screenshot del codice per illustrare il processo di implementazione.

```
[16] pip install mr4mp

     Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
     Requirement already satisfied: mr4mp in /usr/local/lib/python3.8/dist-packages (2.6.2)
     Requirement already satisfied: parts~=1.5 in /usr/local/lib/python3.8/dist-packages (from mr4mp) (1.5.2)

[17] import mr4mp

     from random import choice
     from string import ascii_lowercase
     from timeit import default_timer
```

Figura 2.1 : screenshot del codice

Come in ogni codice prima di tutto installiamo i pacchetti necessari per eseguire il codice. In questo script abbiamo deciso di utilizzare il modulo ‘MR4MP’ per effettuare il calcolo con MapReduce, ed altri 3 moduli quali ‘random’ , ‘string’ e ‘timeit’ ovviamente non abbiamo importato tutto i moduli ma solo le funzioni che sono necessarie.

```
[19] text = text.replace('\n', ' ')
     text = text.replace('-', ' ')
     text = text.split(' ')
```

```
[20] new_d = []
     for list_2 in text:

       list_2 = list_2.replace(',','')
       list_2 = list_2.replace('.','')
       list_2 = list_2.replace(':','')
       list_2 = list_2.replace(';','')
       list_2 = list_2.replace('?','')
       list_2 = list_2.replace('!','')
       list_2 = list_2.lower()
       new_d.append(list_2)
             #print(list_2)
```

Figura 2.2 : screenshot del codice

In queste linee di codice eseguiamo una pulizia del file chiamato ‘text’ che nel nostro esempio è un testo generato casualmente in latino. Vogliamo innanzitutto eliminare tutti quei caratteri speciali che possono ostruire la nostra classificazione o che non riteniamo importanti. In fine decidiamo di creare una lista dove le parole del testo sono tutte in fila e sono separate da una virgola.

```
[21] def map(key):
         return {key:1}

     def reduce(pairs1,pairs2):
         #print("pairs1 keys:", list(pairs1.keys()))
         #print("pairs2 keys:", list(pairs2.keys()))
         keys = pairs1.keys() | pairs2.keys()
         merged = {k: (pairs1.get(k, 0) + pairs2.get(k, 0)) for k in keys}
         return merged
```

Figura 2.3 : screenshot del codice

Ora la parte centrale del nostro codice ovvero la definizione delle funzioni Map e Reduce:
La funzione Map prende una chiave come input e restituisce un dizionario con la chiave come chiave e il valore 1 come valore. Questa funzione viene utilizzata per elaborare i dati in input e produrre un insieme di coppie di valori chiave intermedi.
La funzione Reduce richiede due dizionari (‘pairs1’ e ‘pairs2’) come input, che rappresentano due insiemi di coppie chiave-valore intermedi generati dalla funzione Map. La funzione, quindi, fonde questi due insiemi delle coppie aggiungendo i valori delle coppie con la stessa chiave.
La funzione prima crea un insieme di tutte le chiavi in entrambi i dizionari usando l’operatore dell’unione ‘ | ’. Poi crea un nuovo dizionario iterando sopra le chiavi e sommando i valori per ogni chiave da entrambi i dizionari. Il metodo ‘get’ viene utilizzato per recuperare il valore di una chiave, con un valore predefinito di 0 se la chiave non esiste nel dizionario.
Infine, la funzione restituisce il dizionario unito con i valori combinati per ogni chiave.

```
[22]
start = default_timer()
pool = mr4mp.pool()
#help(pool.mapreduce)
out = pool.mapreduce(map, reduce, new_d)
pool.close()
print("Finished in " + str(default_timer()-start) + "s using " + str(len(pool)) + " process(es).")

Finished in 0.08400946400001885s using 2 process(es).
```

Figura 2.4 : screenshot del codice

Una volta definite le 2 funzioni è molto semplice effettuare il calcolo con MapReduce,
Alla variabile ‘start’ viene assegnato il valore restituito dalla funzione ‘default_timer’, che è una funzione che restituisce l'ora corrente in secondi. Viene utilizzato per registrare l'ora di inizio dell'operazione MapReduce. Alla variabile ‘pool’ viene assegnato il risultato della chiamata della funzione mr4mp.pool(), che crea un pool di processi di lavoro per eseguire l'operazione MapReduce in parallelo. Il numero di processi creati dipende dalle risorse disponibili sul sistema.
Il metodo ‘mapreduce’ dell'oggetto pool viene poi chiamato con gli argomenti ‘map’, ‘reduce’ e ‘new_d’. La funzione ‘map’ è la stessa del blocco di codice precedente e anche la funzione ‘reduce’ è la stessa. L'argomento ‘new_d’ è un dizionario contenente i dati di input che verranno elaborati dalla funzione ‘map’.
Il metodo ‘mapreduce’ esegue il funzionamento di MapReduce in parallelo utilizzando i processi operativi del pool. L'output dell'operazione viene assegnato alla variabile out. Il pool viene quindi chiuso utilizzando il metodo ‘pool.close()’ per rilasciare le risorse utilizzate dai processi di lavoro.
Infine, il tempo totale impiegato dall'operazione MapReduce viene calcolato sottraendo il tempo di inizio registrato nella variabile di inizio dall'ora corrente restituita dalla funzione di default. Il risultato viene stampato sulla console, insieme al numero di processi di lavoro utilizzati nell'operazione.

```
[25] sorted_dict = dict(sorted(out.items()))
     print(sorted_dict)

'amicitia': 1, 'amittere': 1, 'animi': 2, 'appellantur': 1, 'appetitum': 1, 'archytam': 1, 'artis': 1, 'at': 1, 'atque': 2, 'atqui': 1, 'attinet': 1, 'audio'
```

Figura 2.5 : Il risultato di mapreduce per il conteggio di parole

La variabile ‘out’ contiene l'output dell'operazione MapReduce, che è un dizionario con chiavi che rappresentano parole e valori che rappresentano il numero di volte in cui ogni parola appare nei dati di input.

La funzione ‘sorted’ viene chiamata con il metodo ‘items()’ del dizionario ‘out’ come argomento. Questo restituisce un elenco di tuple, dove ogni tupla contiene una coppia chiave-valore dal dizionario out.

La funzione ‘sorted’ ordina quindi questa lista di tuple in base alle chiavi (parole) in ordine crescente.

La funzione ‘dict’ viene chiamata con la lista ordinata di tuple come argomento, che crea un nuovo dizionario con le stesse coppie di valori chiave della lista ordinata di tuple.

Infine, il dizionario ordinato viene assegnato alla variabile ‘sorted_dict’ e quindi stampato sulla console utilizzando la funzione di stampa come in Fig. 2.5.

Nel complesso, questo codice sta ordinando l'output dell'operazione MapReduce in base alle chiavi (parole), che consente di analizzare e interpretare facilmente i risultati.

3. L’algoritmo PageRank in MapReduce

Il terzo capitolo approfondisce l’algoritmo PageRank, che può essere considerato il nucleo di molti motori di ricerca web. L’algoritmo PageRank viene originariamente sviluppato da Larry Page e Sergey Brin come parte del loro lavoro per Google [1]. L’essenza dell’algoritmo PageRank è quella di assegnare un punteggio a ogni pagina sul web , in base all’importanza, al numero e alla rilevanza dei link che puntano verso quella pagina web. L'algoritmo è diventato sempre più complesso nel tempo, infatti grazie al campo della serach engine optimization (SEO) pagine web ed algoritmi sono diventati sempre più sofisticati. In questo capitolo, esploreremo le basi dell'algoritmo PageRank, alcune delle limitazioni e delle sfide che sorgono quando si utilizza questo algoritmo. Discuteremo anche le ottimizzazioni che possono essere fatte per migliorare le prestazioni e la precisione dei calcoli, e applicheremo l'algoritmo ad un grafo di dati del mondo reale per dimostrare la sua applicabilità pratica.

Uno dei più grandi cambiamenti nella società dell'informazione alla fine del secolo scorso fu la possibilità di effettuare una ricerca efficiente ed accurata sul Web grazie ai motori di ricerca come Google.
Quando PageRank fu stabilito come una tecnica essenziale per i motori di ricerca , gli spammer ne abusarono inventando altre tecniche per manipolare il PageRank di una pagina Web, spesso chiamate ‘link spam’. Tutto ciò portò ad uno sviluppo di altre tecniche, come ad esempio il TrustRank per evitare che gli spammer attacchino il PageRank. [3]

3.1 i primi motori di ricerca

C’erano molti motori di ricerca prima di Google. La maggior parte di essi lavorava cercando sul web ed elencando le parole trovate sulla pagina Web in un indice invertito. Un indice invertito
è una struttura di dati che rende facile, dato un termine, trovare tutti i luoghi in cui appare tale termine. Quando viene effettuata una ricerca sul web con un elenco di termini, le pagine che contengono tali termini vengono estratte dall’indice invertito e classificate in una maniera tale da riflettere l’uso dei termini all’interno della pagina. Con l’incremento dell’utilizzo dei motori di ricerca gli spammers videro l’opportunità di alterare il funzionamento dei motori di ricerca per guidare le persone sulle proprie pagine Web.

Pertanto, se una persona vendesse t-shirt sul web, e tutto ciò che gli importasse fosse di attrarre persone sulla sua pagina Web senza riguardo per cosa le persone abbiano realmente cercato. Semplicemente, si potrebbe aggiungere un termine come "film" alla pagina, e farlo migliaia di volte, così un motore di ricerca avrebbe pensato che fossi una pagina terribilmente importante su film. Quando un utente ha effettuato una ricerca con il termine "film", il motore di ricerca elencherebbe la pagina per prima. Per evitare che le migliaia di occorrenze di "film" appaiano sulla pagina, si potrebbe dare lo stesso colore dello sfondo. E se semplicemente aggiungendo "film" alla pagina non ha fatto il trucco, poi si potrebbe andare sul motore di ricerca, cercare "film," e vedere che pagina è tornata come la prima scelta. Poi, copiare quella pagina nella propria pagina, di nuovo utilizzando il colore dello sfondo per renderlo invisibile.

Tecniche per ingannare i motori di ricerca per far credere che la pagina è su qualcosa che non è, sono chiamati 'term spam'. La capacità di tali soggetti di operare così facilmente ha reso i primi motori di ricerca quasi inutili. Per combattere i 'term spam', Google ha introdotto due innovazioni:

1: PageRank è stato utilizzato per simulare dove i navigatori del Web , iniziando in una pagina Web casuale , tendano ad aggregarsi se hanno seguito casualmente i link presenti su quella pagina , e questo processo può essere ripetuto moltissime volte. Così, le pagine Web che hanno un alto numero di navigatori vengono considerate più 'importanti' rispetto a pagine che vengono visitate raramente. Google preferisce una pagina importante rispetto ad una non importante quando deve decidere quale pagina mostrare dopo una ricerca.

2: Il contenuto di una pagina veniva giudicato non solo dai termini che apparivano su quella pagina, ma anche dai termini utilizzati o presenti nei link collegati a quella pagina . Notiamo come sia facile per uno spammer aggiungere termini fasulli sulla pagina che controlla, ma non riesce con tale facilità ad aggiungere quei termini fasulli su una pagina che punta sulla sua pagina ,se essi non controllano pure quelle pagine.

Queste due tecniche insieme rendono molto difficile per l' ipotetico venditore di t-shirt ingannare Google. Mentre il venditore può ancora aggiungere il termine "film" alla sua pagina, il fatto che Google creda a ciò che dicono le altre pagine su di lui rispetto a cosa dice la pagina di sé stessa nega l'utilizzo di termini fasulli. L'ovvia contromisura da parte del venditore può essere quella di creare molte pagine Web con dei link che punto alla sua pagina di t-shirt con i link che connettono le pagine rinominati 'film'. Ma a queste pagine non sarebbe data molta importanza da PageRank, dal momento che altre pagine non avrebbero link a loro. Il venditore di t-shirt potrebbe creare molti link tra le sue pagine, ma nessuno di queste pagine otterrebbero molta importanza secondo l'algoritmo PageRank, e pertanto, ancora non sarebbe in grado di ingannare Google a pensare la sua pagina fosse sui film.

E 'ragionevole chiedersi perché la simulazione di navigatori casuali dovrebbe permetterci di approssimare la nozione intuitiva della "importanza" delle pagine. Ci sono due motivazioni correlate che hanno ispirato questo approccio.

1: Gli utenti del Web ‘votano con i piedi’. Gli utenti tendono ad aggiungere link che portano a pagine buone o utili rispetto a link che portano a pagine inutili.

2: Il comportamento di un navigatore casuale indica quali pagine Web un utente tende a visitare. Gli utenti tendono a visitare molto più spesso pagine utili rispetto a pagine inutili.

Al di là delle ragioni di fondo, il buon funzionamento di PageRank è stato dimostrato empiricamente, e pertanto merita di essere studiato dettagliatamente . [3]

3.2 Il modello PageRank

PageRank è una funzione che assegna un numero reale ad ogni pagina Web (o almeno alla porzione di Web che si prende in considerazione). L’intuizione è quella che più alto è il valore di PageRank di una pagina più ‘importante’ è questa pagina. Non esiste una formulazione univoca per l’algoritmo PageRank, infatti esistono variazioni della formulazione base che possono alterare il punteggio assegnato dal PageRank originario. Iniziamo definendo il modello di base e idealizzato di PageRank per poi aggiungerci delle modifiche necessarie per affrontare problemi del mondo reale legati alla struttura del web.

Immaginiamo il web come un grafo orientato, dove le pagine Web sono i vertici ed i link che collegano le pagine sono gli archi. Il termine grafo orientato in informatica viene usato per indicare una particolare struttura discreta o geometrica, costituita da un insieme finito di punti, detti nodi (oppure vertici), e archi orientati (o segmenti o spigoli), che congiungono coppie di nodi . [3]

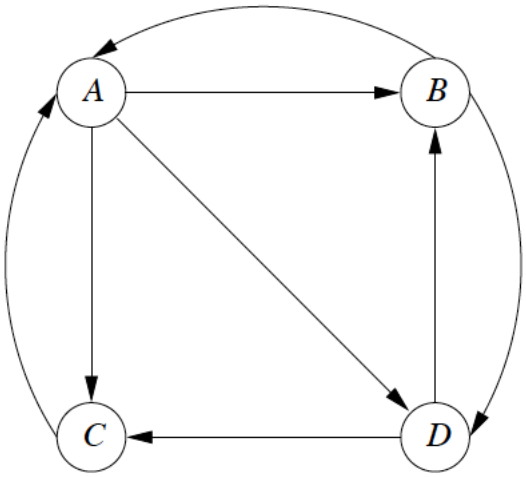

figura 3.1: un esempio ipotetico del Web

La figura 3.1 può essere considerata come un piccolo esempio del web dove abbiamo solo quattro pagine. La pagina A ha dei link che puntano a tutte le altre tre pagine; La pagina B ha solo due link verso A e verso D; La pagina C ha un solo link che punta verso A, e la pagina D ha solamente due link verso B e verso C.

Immaginiamo che nella figura 3.1 un navigatore casuale si trovi sulla pagina A . Ci sono link verso B, C, e D, quindi questo navigatore sarà in futuro su ognuna di queste pagine con probabilità 1/3 e ha una probabilità pari a zero di trovarsi su A. Un navigatore casuale che si trova in B ha, alla prossima iterazione , probabilità pari a 1/2 di trovarsi in A, probabilità pari ad 1/2 di trovarsi in D , e probabilità pari a 0 di trovarsi in B o C.

Generalmente, definiamo la 'matrice di transizione del Web' per descrivere probabilisticamente cosa succede ai navigatori casuali dopo una iterazione. Questa matrice ha dimensione N x N , dove N è il numero di pagine Web che si ha preso in considerazione. L'elemento M_{ij} nella riga i e nella colonna j ha come valore 1/k se la pagina j ha k archi uscenti, e se uno di questi archi punta alla pagina i. Altrimenti l'elemento M_{ij} ha valore pari a 0.

$$M = \begin{bmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Figura 3.2 : la matrice di transizione del Web della figura 3.1

Nella matrice in figura 3.2 , l'ordine delle pagine espresso è quello naturale, ovvero A,B,C e D.

La prima colonna mostra che, come già discusso in precedenza, il navigatore se si trova in A ha probabilità pari ad 1/3 di trovarsi alla prossima iterazione in una di quelle pagine B,C e D. La terza colonna mostra come se il navigatore si trova in C è certo di trovarsi in A alla prossima iterazione.

La distribuzione di probabilità della posizione di un navigatore casuale può essere descritta da un vettore colonna la cui componente j-esima è la probabilità che il navigatore si trovi alla pagina J. Questa probabilità è la funzione PageRank idealizzata.

Supponiamo che iniziamo a navigare con un navigatore casuale in qualunque delle N pagine con la stessa probabilità. Allora il vettore iniziale $\boldsymbol{V}_0$ avrà 1/N per ognuno dei suoi elementi. Se definiamo M come la matrice di transizione del web , allora dopo un'iterazione , la distribuzione del navigatore sarà $M\boldsymbol{V}_0$, dopo due iterazioni sarà $M(M\boldsymbol{V}_0) = M^2\boldsymbol{V}_0$, eccetera . Generalmente , moltiplicando il

vettore iniziale $\boldsymbol{V}_0$ per M un i-esimo numero di volte otterremo la distribuzione del navigatore dopo le iterazioni.

Per capire perché moltiplicando la distribuzione del vettore **V** per M ci dà la distribuzione $x = M\boldsymbol{V}$ alla prossima iterazione, ragioniamo nella maniera seguente. La probabilità X_i che un navigatore casuale si trovi alla pagina 'i' alla prossima iterazione , è $\sum M_{ij}\boldsymbol{V}_j$ per i valori di j. M_{ij} è la probabilità che un navigatore che si trova sulla pagina j sarà alla pagina i dopo la prossima iterazione (nel web solitamente è 0 perché le pagine web sono ancora molto sconnesse fra di loro) e $\boldsymbol{V}_j$ è la probabilità che il navigatore si trovasse alla pagina j all'iterazione precedente.

Questo tipo di comportamento è un esempio della teoria classica dei processi di Markov. È noto che la distribuzione del navigatore si avvicina ad una distribuzione limite **V** che soddisfa la condizione $\boldsymbol{V} = M\boldsymbol{V}$, purché siano soddisfate due condizioni :

1: Il grafo è fortemente connesso; ovvero, è possibile spostarsi da qualunque nodo a qualsiasi altro nodo.

2: non ci siano vicoli cechi : ovvero nodi che non hanno archi uscenti . [3]

Da notare che la figura 3.1 soddisfa queste due condizioni.

Il limite viene raggiunto quando moltiplicando la distribuzione per M un'altra volta , la distribuzione non cambia. In altre parole, il limite **V** è un autovettore di M (un autovettore della matrice M è un vettore $\boldsymbol{v}$ che soddisfa la condizione $M\boldsymbol{v} = l\boldsymbol{v}$ per qualche costante λ, detta autovalore di M). Poiché M è stocastica, ovvero gli elementi all'interno di una colonna sono non negativi e se sommati danno 1, **v** è l'autovettore principale (il suo autovalore associato è il più grande fra tutti gli autovalori) come conseguenza del Teorema di Perron-Frobenius. È da notare inoltre come poiché M è stocastica, l'autovalore associato al autovettore principale è 1.

L'autovettore principale di M ci dice dove molto probabilmente si trovi il navigatore dopo un lungo periodo di tempo. Ricordiamo che l'intuizione dietro PageRank è che più un navigatore è probabile che sia su una pagina, più importante è la pagina. Possiamo calcolare l'autovettore principale di M partendo dal vettore iniziale $\boldsymbol{v}_0$ e moltiplicando per M un certo numero di volte, fino a quando il vettore che otteniamo mostra pochi segni di cambiamento dopo ogni iterazione. In pratica, per il Web stesso, sono sufficienti 50-75 iterazioni per convergere entro i limiti di errore accettabili.

Per il nostro esempio in figura 3.1 , siccome ci sono quattro pagine , il vettore iniziale avrà quattro componenti ognuna con valore 1/4 . La sequenza di approssimazioni al limite che otteniamo moltiplicando ad ogni iterazione per M è:

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 9/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix}, \begin{bmatrix} 15/48 \\ 11/48 \\ 11/48 \\ 11/48 \end{bmatrix}, \begin{bmatrix} 11/32 \\ 7/32 \\ 7/32 \\ 7/32 \end{bmatrix}, \ldots, \begin{bmatrix} 3/9 \\ 2/9 \\ 2/9 \\ 2/9 \end{bmatrix}$$

Figura 3.3 : sequenza del vettore dopo un certo numero di iterazioni.

Vediamo come la probabilità di A è 3/9 oppure equivalentemente a 1/3, mentre quella di B,C e D è 2/9. Questa differenza di probabilità non è grande. Ma nel Web reale, con miliardi di pagine di importanza molto variabile, la vera probabilità di essere in una pagina come www.amazon.com è di ordini di grandezza molto superiori alle pagine tipiche. [3]

3.3 Limiti e svantaggi nella formulazione base dell'algoritmo PageRank

Ricordiamo che una delle condizioni nella teoria dei processi di Markov è l'assenza di vicoli cechi o anche chiamati 'dead-ends', questi vicoli cechi sono pagine che non hanno archi uscenti, ma solamente archi entranti. Se noi permettessimo ai vicoli ciechi di esistere, la nostra matrice di transizione del Web non sarebbe più stocastica , poiché alcune delle colonne non sommano più ad 1. Una matrice di cui la maggior parte delle colonne somma ad 1 viene chiamata sub-stocastica. Se noi provassimo ora a calcolare il nostro PageRank su una matrice sub-stocastica, allora alcuni o tutti i componenti del vettore diventeranno 0. Questo significa che l'importanza fuoriesce dal Web, e non riusciamo ad ottenere alcuna informazione riguardante l'importanza relativa delle pagine.

$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Figura 3.4 : La matrice nella figura 3.2 è stata modificata, trasformando la pagina C in un vicolo cieco

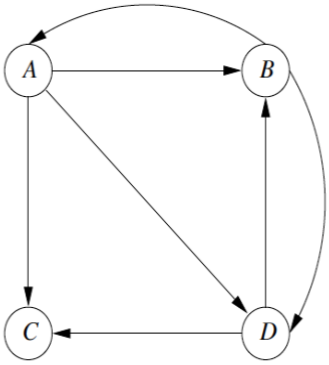

Figura 3.5 : rappresentazione grafica della nostra nuova matrice in figura 3.4

Notiamo come appunto la nostra matrice non è più stocastica ma sub-stocastica poiché la terza somma a 0 e non a 1. Di seguito mostriamo la sequenza di vettori dopo il prodotto per M, il vettore ha sempre come componenti 1/4 poiché le pagine rimangono sempre quattro.

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 3/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{bmatrix}, \begin{bmatrix} 5/48 \\ 7/48 \\ 7/48 \\ 7/48 \end{bmatrix}, \begin{bmatrix} 21/288 \\ 31/288 \\ 31/288 \\ 31/288 \end{bmatrix}, \ldots, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Figura 3.6 : sequenza di vettori dopo il prodotto con M

Come possiamo vedere, la probabilità di un navigatore di trovarsi ovunque diventa 0, in conseguenza all'aumento del numero di iterazioni effettuate. Ci sono due tipi di approcci per gestire i vicoli cechi. [3]

1: possiamo eliminare i vicoli ciechi dal grafo , eliminando anche gli archi entranti al vicolo cieco. Facendo ciò c'è la possibilità di creare nuovi vicoli ciechi che devono a loro volta essere eliminati. Tuttavia, con questo metodo eventualmente arriveremo ad avere un grafo fortemente connesso senza vicoli ciechi.

2: Possiamo modificare il processo di come il navigatore casuale naviga e si sposta sul Web. Questo approccio a cui ci riferiremo come 'tassazione' risolve anche altri problemi della formulazione base di PageRank.

Il secondo problema nella formulazione base di PageRank sono le cosiddette ‘trappole per ragni’, vengono chiamate così perché i programmi che scansionano il Web, registrano le pagine ed i link vengono spesso chiamati ‘ragni’. Una volta che un ragno entra in una trappola per ragni, non riesce più a scappare. Una ‘trappola per ragni‘ sono dei nodi senza vicoli ciechi che però non hanno archi uscenti verso altri nodi.

Possiamo creare una ‘trappola per ragni’ semplicemente dalla nostra figura 3.4 aggiungendo un arco che da C punta in C in questa maniera :

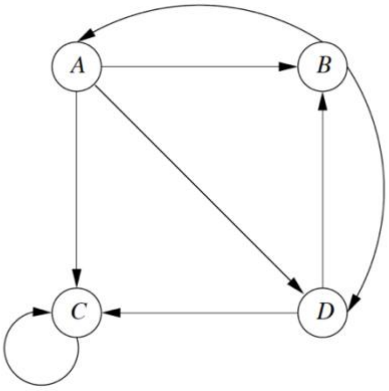

Figura 3.7 : un grafo dove il nodo C è una ‘trappola per ragni’.

$$M = \begin{bmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 1 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{bmatrix}$$

Figura 3.8 : la matrice di transizione del Web per la figura 3.7

Se effettuiamo le classiche iterazioni per calcolare il nostro PageRank sui nodi in figura 3.7 otteniamo :

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 3/24 \\ 5/24 \\ 11/24 \\ 5/24 \end{bmatrix}, \begin{bmatrix} 5/48 \\ 7/48 \\ 29/48 \\ 7/48 \end{bmatrix}, \begin{bmatrix} 21/288 \\ 31/288 \\ 205/288 \\ 31/288 \end{bmatrix}, \ldots, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Figura 3.9 : La sequenza di vettori dopo le iterazioni sulla matrice in figura 3.8

Come potevamo aspettarci tutto il nostro PageRank è concentrato in C, poiché quando un navigatore casuale entra in C non riesce più ad uscire.

Per risolvere il problema dei vicoli cechi e delle 'trappole per ragni' modifichiamo il nostro calcolo di PageRank , ora al nostro navigatore casuale gli aggiungiamo una piccola probabilità di *teletrasporto* ad una pagina casuale , piuttosto che seguire un link uscente dalla loro pagina corrente. Quando vogliamo calcolare il nuovo vettore stimato per il PageRank **V'** dall'attuale PageRank stimato **V** e la matrice di transizione M è :

$$v' = \beta M v + (1-\beta)e/n$$

Dove β è una costante fissa che scegliamo , solitamente compresa tra 0.8 e 0.9, 'e' è un vettore le cui componenti sono tutte 1 con l'appropriato numero di componenti, 'n' è il numero di nodi/pagine presenti nel nostro grafo Web. Il termine $\beta M v$ rappresenta la situazione in cui , con probabilità pari a β, il navigatore casuale decide di seguire un link uscente dalla pagina attuale. Il termine $(1-\beta)e/n$ è un vettore le cui componenti hanno come valore $(1-\beta)/n$ e questo rappresenta , con probabilità pari ad 1- β , la possibilità di un navigatore casuale di teletrasportarsi su una pagina casuale.
Da notare come se il grafo non ha vicoli cechi , allora la probabilità di introdurre un nuovo navigatore casuale sarà esattamente uguale alla probabilità che il navigatore casuale decida di non seguire un link uscente dalla pagina attuale. In questo caso, è ragionevole visualizzare il navigatore casuale come se fosse una decisone tra seguire un link o teletrasportarsi a caso. Tuttavia, se ci sono dei vicoli cechi , allora c'è una terza possibilità , dove il navigatore non va da nessuna parte. Siccome il termine $(1-\beta)e/n$ non dipende dalla somma delle componenti del vettore **V**, allora saranno sempre presenti delle frazioni di un navigatore che opera sul Web. Cioè, quando ci sono dei vicoli ciechi, la somma delle componenti di **V** può essere inferiore a 1, ma non raggiungerà mai 0.
Vediamo ora come con l'introduzione della 'tassazione' cambia il calcolo del nostro PageRank per la figura 3.6. Utilizziamo un β pari a 0.8 allora l'equazione per le iterazioni diventa :

$$\mathbf{v}' = \begin{bmatrix} 0 & 2/5 & 0 & 0 \\ 4/15 & 0 & 0 & 2/5 \\ 4/15 & 0 & 4/5 & 2/5 \\ 4/15 & 2/5 & 0 & 0 \end{bmatrix} \mathbf{v} + \begin{bmatrix} 1/20 \\ 1/20 \\ 1/20 \\ 1/20 \end{bmatrix}$$

Figura 3.10 : L'equazione per calcolare il PageRank per la figura 3.7 dopo aver introdotto il concetto di 'tassazione'

Abbiamo incorporato il termine β in M moltiplicando ogni elemento per 4/5. Le componenti del vettore $(1-\beta)e/n$ sono 1/20 , poiché 1- β = 1/5 e n = 4. Mostriamo di seguito le prime iterazioni per poi mostrare il vettore finale.

$$\begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}, \begin{bmatrix} 9/60 \\ 13/60 \\ 25/60 \\ 13/60 \end{bmatrix}, \begin{bmatrix} 41/300 \\ 53/300 \\ 153/300 \\ 53/300 \end{bmatrix}, \begin{bmatrix} 543/4500 \\ 707/4500 \\ 2543/4500 \\ 707/4500 \end{bmatrix}, \ldots, \begin{bmatrix} 15/148 \\ 19/148 \\ 95/148 \\ 19/148 \end{bmatrix}$$

Figura 3.11 : La sequenza di vettori dopo diverse iterazioni

Come possiamo notare siccome C è una 'trappola per ragni' ha assorbito più della metà del nostro PageRank per sé stessa. Tuttavia, gli effetti sono stati arginati ed anche le altre pagine riescono ad assorbire una parte del PageRank. [3]

3.4 Ottimizzazione dell'implementazione di MapReduce nell'algoritmo PageRank

Per calcolare il PageRank di un grande grafo che rappresenta il Web, dobbiamo effettuare un prodotto scalare tra la nostra matrice di transizione ed il nostro vettore **V** più o meno 50 volte, finche il vettore converge o presenta delle differenze minime rispetto all'iterazione precedente. Tuttavia, quando vogliamo effettuare tale calcolo su un grafo di dimensione elevate incorriamo in dei problemi.

La matrice di transizione del Web è molto sparsa, ciò significa che abbiamo molti elementi della matrice con un valore pari a 0, quindi rappresentare la matrice con tutti i suoi elementi è molto inefficiente. Piuttosto, preferiamo rappresentare la matrice con i suoi elementi diversi da 0.
Il modo corretto per rappresentare qualsiasi matrice sparsa è elencare le posizioni degli elementi diversi da zero ed i loro rispettivi valori. Se utilizziamo numeri interi da 4-byte per elencare la posizione degli elementi diversi da 0 e numeri in virgola mobile a doppia precisione da 8 byte per il corrispettivo valore, allora servono 16 bytes per ogni elemento diverso da 0.

Una iterazione dell'algoritmo PageRank richiede di utilizzare un vettore PageRank $\boldsymbol{V}$ stimato per calcolare il prossimo vettore stimato $\boldsymbol{V'}$ seguendo la seguente equazione:

$$v' = \beta M v + (1 - \beta) e / n$$

Ricordiamo che $\boldsymbol{\beta}$ è una costante leggermente inferiore ad 1, $\boldsymbol{e}$ è un vettore di tutti 1 , e che $\boldsymbol{n}$ è il numero di nodi presente nel grafo che la matrice di transizione M rappresenta.
Per implementare il nostro modello MapReduce dobbiamo scrivere due funzioni, la funzione Map e la funzione Reduce. Se $\boldsymbol{n}$ fosse abbastanza piccolo in maniera tale che ogni lavoro della funzione Map riuscisse ad immagazzinare tutto il vettore $\boldsymbol{V}$ nella memoria principale , e se inoltre ci fosse

abbastanza spazio per il vettore risultante $\boldsymbol{V'}$ allora ciò di cui stiamo parlando è una semplice moltiplicazione matrice-vettore. Il passo seguente è quello di moltiplicare gli elementi di M per $\boldsymbol{\beta}$ ed aggiungerci $(1-\beta)/n$ ad ogni elemento. In realtà risulta facile pensare come, data la grandezza del Web al giorno d'oggi $\boldsymbol{V}$ risulti troppo grande per essere immagazzinato nella memoria principale.
Possiamo attuare diverse strategie per risolvere questo problema, una di quelle che risulta più efficiente è quella di partizionare la matrice in K^2 quadrati, mentre il vettore viene diviso in K strisce . [3]

$$\begin{bmatrix} \mathbf{v}_1' \\ \mathbf{v}_2' \\ \mathbf{v}_3' \\ \mathbf{v}_4' \end{bmatrix} = \begin{bmatrix} M_{11} & M_{12} & M_{13} & M_{14} \\ M_{21} & M_{22} & M_{23} & M_{24} \\ M_{31} & M_{32} & M_{33} & M_{34} \\ M_{41} & M_{42} & M_{43} & M_{44} \end{bmatrix} \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \\ \mathbf{v}_4 \end{bmatrix}$$

Figura 3.12 : un esempio di una matrice divisa in quadrati

In figura 3.12 vediamo il risultato di una matrice 4x4 divisa per K = 4. Gli step in cui moltiplichiamo per $\boldsymbol{\beta}$ ed aggiungiamo $(1-\beta)/n$ non vengono mostrati perchè sono semplici e diretti, indipendentemente dalla strategia che utilizziamo.
Con questa strategia, utilizziamo K^2 attività Map. Ogni attività riceve un quadrato della matrice M, per esempio M_{ij}, e una striscia del vettore $\boldsymbol{V}$, che deve essere $\boldsymbol{V_j}$. Notiamo come ogni striscia del vettore viene inviata a K differenti attività Map ; $\boldsymbol{V_j}$ viene inviato all'attività per gestire M_{ij} per ognuno dei k possibili valore di i. Cosi, $\boldsymbol{V}$ viene trasmesso k volte sulla nostra rete. Però, ogni quadrato della matrice viene trasmesso una volta sola. Per le ragioni precedenti ci aspettiamo che le dimensioni della matrice siano notevolmente maggiori rispetto a quelle del vettore, allora i costi di trasmissione sono leggermente maggiori del minimo possibile , e siccome effettuiamo una gran parte del lavoro nelle attività Map riusciamo a salvare le informazioni quando passano dalla funzione Map alla funzione Reduce.
Il vantaggio di questo approccio è che riusciamo a tenere sia la j-esima striscia di $\boldsymbol{V}$ sia la i-esima striscia di $\boldsymbol{V'}$ nella memoria principale mentre elaboriamo M_{ij}. Tutti i termini generati da M_{ij} e $\boldsymbol{V_j}$ contribuiscono solamente a $\boldsymbol{V_i'}$ e a nessun'altra striscia di $\boldsymbol{V'}$ **.** [3]

3.5 Computazione effettiva dell'algoritmo PageRank su un grafo reale

Vogliamo ora testare un possibile algoritmo PageRank con MapReduce su un grafo di dati che simula la realtà, per dimostrare l'efficienza del modello MapReduce sul calcolo del PageRank e per la sua generale efficacia a gestire enormi set di dati. Il nostro set di dati [7] è stato preso gentilmente da [8] anche conosciuto come 'Stanford Large Network Dataset Collection' (SNAP).

Iniziamo in nostro codice importando tutti i moduli necessari e caricando i nostri dati per salvarli in una variabile.

```
[ ] import numpy as np
    import networkx as nx
    import matplotlib.pyplot as plt
    import mr4mp
    import itertools
    from random import choice
    from string import ascii_lowercase
    import math
```

```
[ ] with open("/content/web-Stanford (1).txt") as f:
        # read the file contents into a list of strings
        lines = f.readlines()
```

Figura 3.13 : screenshot del codice

```
[ ] graph = nx.DiGraph()
    for line in t_lines:
        edge = line.split()
        node1 = int(edge[0])
        node2 = int(edge[1])
        graph.add_edge(node1, node2)

    # Print some basic information about the graph
    print("Number of nodes:", graph.number_of_nodes())
    print("Number of edges:", graph.number_of_edges())

    nodes = sorted(list(graph.nodes()))
    edges = sorted(list(graph.edges()))
```

Figura 3.14 : screenshot del codice

Questo codice crea un grafo diretto usando la libreria 'networkx', itera attraverso una raccolta di dati in input che rappresentano gli archi nel grafico, aggiunge ogni arco al grafo e successivamente riporta il numero di nodi e archi nel grafo risultante.

```
out_degree = dict(graph.out_degree())


for node in out_degree.keys():
    if out_degree[node] > 0:
        out_degree[node] = 1 / out_degree[node]
    else:
        out_degree[node] = 0


nodes = sorted(out_degree.keys())
node_indices = {node: i for node, i in enumerate(nodes)}


rni = {k:v for v,k in node_indices.items()} #reverse node index
```

Figura 3.15 : screenshot del codice

In queste righe di codice andiamo prima a calcolarci il cosiddetto ‘outdegree’ (grado uscente) di ogni nodo, per poi calcolarci il reciproco di questa misura. Facciamo ciò contando verso quante pagine il nodo i punta e creiamo un dizionario dove le chiavi sono i nodi ed i valori associati sono il reciproco del ‘outdegree’ se esso è maggiore di 0 altrimenti il valore è 0. Successivamente andiamo a creare altri due dizionari molto utili per le misure di centralità ed il nostro PageRank , il primo dizionario rappresenta una ‘hash-map’ per mappare l’ID di ogni nodo con una riga della nostra matrice , il secondo è il reciproco di questo dizionario dove le chiavi sono l’ID dei nodi ed i valori sono la corrispettiva riga della nostra matrice.

```
n = len(nodes)
matrix = np.zeros((n, n))
```

Figura 3.16 : screenshot del codice

In questo codice andiamo ad inizializzare la nostra matrice di dimensioni (NxN) con tutti zero al suo interno.

```
for edge in edges:
  if edge in edges:
    value = out_degree[edge[0]]
    row = rni[edge[1]]
    column = rni[edge[0]]
    matrix[row][column] = value


final_matrix = (matrix * 0.8) + 1/len(matrix)
```

Figura 3.17 : screenshot del codice

In queste righe iteriamo sulla lista degli archi presenti nei nostri dati, effettuiamo un doppio controllo per verificarne la reale esistenza per poi salvarci il valore del ‘outdegree’ di quel nodo nella variabile ‘value’. Poi andiamo a recuperare il valore della riga e della colonna di quel nodo grazie al nostro dizionario creato precedentemente. In seguito, andiamo ad aggiungere l’elemento della matrice corrispondente alla riga ed alla colonna con valore uguale alla variabile ‘value’ ovvero l’outdegree di quel nodo.

Una volta fatto ciò abbiamo creato la nostra matrice di transizione, necessaria per il calcolo di PageRank, effettuiamo subito i 2 step più semplici ovvero moltiplichiamo la matrice per un fattore di smorzamento in questo caso impostato a 0.85 per poi aggiungerci (1-0.85)/n dove n rappresenta il numero di righe o colonne nella matrice. Una volta finiti questi passaggi la nostra matrice di transizione è pronta per essere scomposta ed utilizzata per il calcolo di PageRank.

```
def transition(matrix):
  li = []
  for i, line in enumerate(matrix):
    for j, element in enumerate(line):
      li.append([(i,j,element)])
  return li
c = transition(final_matrix)
for element in c:
  element.append((element[0][0],1/len(matrix)))

def update_matrix(matrix, vector):
  for line in matrix:
    for k in vector:
      if line[0][0] == k[0]:
        line[1] = k
```

Figura 3.18 : screenshot del codice

In queste line di codice definiamo due funzioni molto importanti che servono per creare una lista di liste, dove in ogni lista sono presenti due tuple, la prima tupla rappresenta la riga, la colonna e l'elemento corrispondente della matrice. La seconda tupla rappresenta il nostro vettore pesi che viene inizializzato con valore 1/n dove n è sempre il numero di nodi presenti nel grafo, per poi aggiornare il secondo elemento della seconda tupla dopo ogni iterazione con il valore del PageRank calcolato per quella iterazione.

In queste righe abbiamo deciso di applicare la strategia dove inviamo n volte il vettore pesi alla nostra matrice; quindi, abbiamo degli svantaggi riguardanti i costi di trasmissione però questo è un trade-off per avere dei vantaggi in termini di memoria principale utilizzata.

```
def mapp(block):
  #print(block)
  if block[0][0] == block[1][0]:
    t ={(block[0][0],block[0][1]): block[0][2]*block[1][1]}

    return{(block[0][0],block[0][1]): block[0][2]*block[1][1]}

def reducer(pairs1,pairs2):
    #print("pairs1 keys:", list(pairs1.keys()))
    #print("pairs2 keys:", list(pairs2.keys()))
    keys = pairs1.keys() | pairs2.keys()
    #print('pair1',pairs1)
    #print('pair2', pairs2)
    merged = {k: (((pairs1.get(k, 0) + pairs2.get(k, 0)))) for k in keys}
    #print(keys)
    #print(merged)
    return merged
```

Figura 3.19 : screenshot del codice

Una volta preparati i nostri dati e la nostra matrice di transizione non resta che definire le funzioni Map e Reduce.

Nel codice, 'Mapp(block)' è chiamato dal framework MapReduce per ogni blocco della matrice e del vettore in input. L'output di ogni chiamata a 'Mapp(block)' è un dizionario contenente una coppia chiave-valore, che rappresenta un singolo elemento nella matrice in output. Le chiavi di questi dizionari sono tuple che rappresentano gli indici di riga e colonna della matrice di output e i valori sono i prodotti degli elementi corrispondenti nella matrice e nel vettore in input.

Dopo il passaggio Map, il framework chiama la funzione ‘reducer(pairs1,pairs2)’ per unire i dizionari prodotti dal passaggio Map. La funzione Reduce accetta due dizionari come input e restituisce un dizionario unito che contiene la somma dei valori per ogni chiave in entrambi i dizionari in input. Questo passaggio combina i risultati del passaggio Map in un unico dizionario, che rappresenta la matrice output finale.

```
from timeit import default_timer

start = default_timer()
counter = 0
while counter < 50:
  pool = mr4mp.pool()
  out = pool.mapreduce(mapp,reducer, c)
  pool.close()
  print('Iteration number: '+str(counter)+" finished in " + str(default_timer()-start) + "s using " + str(len(pool)) + " process(es).")
  result = {}
  for key, value in out.items():
    if key[0] not in result:
        result[key[0]] = value
    else:
        result[key[0]] += value
  sum_pr = sum(x[1] for x in result.items())
  output = [(x[0], x[1]/sum_pr) for x in result.items()]
  update_matrix(c, output)
  counter += 1
print('this is the final pangeRank'+str(output))
```

Figura 3.20 : screenshot del codice

Queste linee di codice eseguono un ciclo per 50 iterazioni. All'interno di ogni iterazione, viene creato un nuovo ‘pool multiprocessing’ usando la libreria mr4mp. La funzione ‘pool.mapreduce()’ applica le funzioni ‘Mapp’ e ‘reducer’ all'input c, che contiene la matrice di transizione del grafo. L'output di ‘pool.mapreduce()’ viene memorizzato nella variabile out.

Dopo la fase di mapreduce, il pool viene chiuso e l'output viene elaborato per calcolare i valori finali di PageRank. L'output è un dizionario ed i valori sono sommati per ogni chiave che ha lo stesso indice di nodo. Quindi, viene calcolata la somma di tutti i valori PageRank e ogni valore PageRank viene diviso per questa somma, ovvero effettuiamo una normalizzazione dei risultati. L'elenco risultante delle tuple, contenente gli indici dei nodi e i corrispondenti valori di PageRank, che vengono utilizzati per aggiornare la matrice di transizione c utilizzando la funzione ‘update_matrix’ ovvero la funzione di aggiornamento della matrice.

Il ciclo continua per 50 iterazioni e i valori PageRank finali vengono stampati alla fine del ciclo. Vengono inoltre stampati il tempo di esecuzione e il numero di processi utilizzati in ogni iterazione.

```
Iteration number: 40 finished in 181.71003902100028s using 2 process(es).
Iteration number: 41 finished in 187.17535311600022s using 2 process(es).
Iteration number: 42 finished in 191.05397974300013s using 2 process(es).
Iteration number: 43 finished in 194.88852400099995s using 2 process(es).
Iteration number: 44 finished in 200.41427883400002s using 2 process(es).
Iteration number: 45 finished in 204.26145918700013s using 2 process(es).
Iteration number: 46 finished in 208.15239434700015s using 2 process(es).
Iteration number: 47 finished in 213.34128223800008s using 2 process(es).
Iteration number: 48 finished in 217.4561114170001s using 2 process(es).
Iteration number: 49 finished in 221.25825041100006s using 2 process(es).
this is the final pangeRank[(71, 3.025431370024346e-108), (29, 3.02543137002439e-108), (48, 3.0254313700243772e-108), (6,
```

Figura 3.21 : un possibile output del nostro codice.

In figura 3.21 possiamo vedere come sarà un possibile output del nostro codice.

Con queste linee abbiamo terminato il nostro codice per calcolare il PageRank in un grafo, siccome il nostro grafo contiene oltre centomila nodi una rappresentazione grafica sarebbe inefficiente, perciò, ora mostriamo una visualizzazione grafica del nostro grafo su un insieme ristretto di nodi , i primi mille.

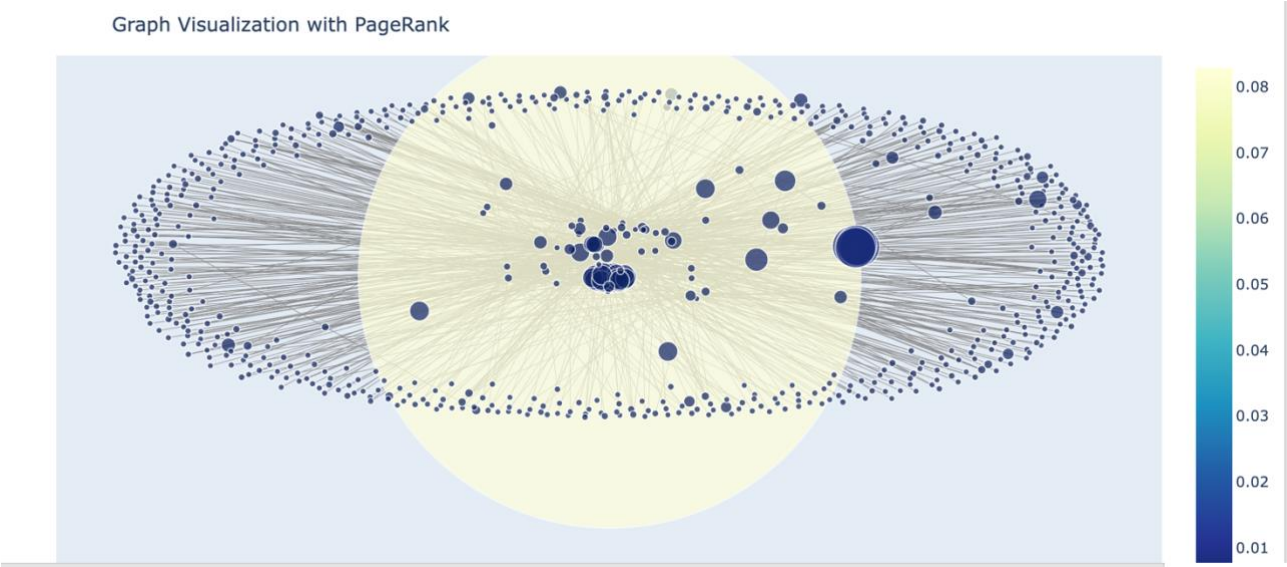

Figura 3.22 : una rappresentazione grafica dei nostri risultati PageRank

Come possiamo vedere dalla figura 3.22, se calcoliamo il PageRank dai nostri dati sui primi mille nodi vediamo come la maggior parte dei nodi ha un valore abbastanza basso mentre invece è presente un nodo che assorbe quasi tutto il punteggio PageRank.
Visualizzare un grafo con PageRank può aiutarci a capire meglio l'importanza e la centralità dei nodi all'interno della rete. PageRank è una misura dell'importanza di un nodo in base alle sue connessioni ad altri nodi importanti, e queste informazioni possono essere utilizzate per identificare i player chiave o le comunità all'interno della rete[9]. Visualizzando il grafo con PageRank, possiamo vedere quali nodi hanno il rango più alto e sono quindi i più influenti o centrali, e possiamo anche vedere come questi nodi sono collegati ad altri nodi all'interno della rete. Questo può aiutarci a identificare modelli o cluster importanti all'interno della rete, nonché potenziali punti deboli o vulnerabilità che possono essere sfruttati. In definitiva, la visualizzazione di un grafo con PageRank può fornire preziose informazioni sulla struttura e le dinamiche di reti complesse, e ci aiutano a prendere decisioni meglio informate sulla base dei dati.

Conclusioni

In questa tesi, abbiamo esplorato il modello MapReduce, che è emerso come soluzione popolare per l'elaborazione di grandi collezioni di dati nell'era dei Big Data. Abbiamo iniziato con un'introduzione al modello, fornendo una breve storia e una panoramica della sua architettura. Abbiamo inoltre discusso le fasi chiave del calcolo in MapReduce, tra cui le fasi Map, Shuffle e Reduce.

Dopo aver introdotto i concetti base di MapReduce, abbiamo poi approfondito due applicazioni specifiche del modello. La prima applicazione che abbiamo esplorato è stato il problema del conteggio delle parole in un documento. Abbiamo discusso l'importanza di questo compito in vari campi, tra cui l'elaborazione del linguaggio naturale e il recupero delle informazioni. Abbiamo quindi presentato un'implementazione del conteggio delle parole utilizzando MapReduce, mostrando come il framework può elaborare in modo efficiente file di testo di grandi dimensioni e generare la frequenza assoluta delle parole.

La seconda applicazione che abbiamo esplorato è stato l'algoritmo PageRank, che è ampiamente utilizzato tuttora nei motori di ricerca web per ordinare le pagine in base alla loro rilevanza. Abbiamo discusso il principio di funzionamento alla base dell'algoritmo PageRank e le sfide associate al calcolo dei relativi punteggi in modo efficiente. Abbiamo quindi presentato un'implementazione di PageRank utilizzando MapReduce, dimostrando come il framework può essere utilizzato per elaborare grafi di grandi dimensioni e calcolare in modo efficiente i punteggi PageRank.

Nel complesso, questa tesi ha dimostrato la versatilità del framework MapReduce come strumento computazionale per l'elaborazione di grandi set di dati. Mentre ci sono limitazioni e sfide associate a MapReduce, come la necessità di un'attenta partizione dei dati e i potenziali colli di bottiglia nella fase di rimescolamento e invio dei record, i vantaggi della scalabilità, la tolleranza ai guasti e l'elaborazione parallela rendono MapReduce una soluzione interessante per le organizzazioni che lavorano con i Big Data.

Guardando al futuro, ci aspettiamo che MapReduce continui a rappresentare uno schema di calcolo indispensabile e ad essere una tecnologia importante nel campo dei Big Data, anche se recentemente sono emersi nuovi framework e strumenti. I principi e i concetti alla base di MapReduce, come l'elaborazione distribuita e la tolleranza dei guasti, sono stati adottati anche in questi strumenti più moderni, e sono fondamentali per le sfide poste dall’elaborazione e l’analisi di grandi dati e rimarranno rilevanti per i prossimi anni.

Bibliografia

[7] Leskovec, J., Adamic, L. A., Huberman, B. A. (2009) Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. Communications of the ACM,vol. 50, no. 12, pp. 135-141.

[5] Carbone, P. (2015) Apache Flink: Stream and Batch Processing in a Single Engine. IEEE Data Engineering Bulletin, vol. 38, no. 4, pp. 28-38.

[1] Dean, J., Ghemawat, S. (2010) MapReduce: Simplified Data Processing on Large Clusters. In: Dean, J., Ghemawat, S. (eds.), Hadoop: The Definitive Guide, O'Reilly Media, Sebastopol, pp. 38-46.

[2] Lin, J., Dyer, C. (2010) ta-Intensive Text Processing with MapReduce. In: Lin, J., Dyer, C. (eds.), Data-Intensive Text Processing with MapReduce, Morgan & Claypool Publishers, San Rafael, pp. 1-165.

[4] White, T. (2012) Hadoop: The Definitive Guide. O'Reilly Media, Sebastopol.

[3] Leskovec, J., Rajaraman, A., Ullman, J. D. (2014) Mining of Massive Datasets. Cambridge University Press.

[9] Newman, M. E. J. (2018). Networks: An Introduction. Oxford University Press.

Sitografia

[6] Apache Software Foundation (2023) Apache Flink. Available at: https://flink.apache.org/ (Accessed: March 16, 2023).

[8] Stanford University (2023) SNAP Datasets. Available at: http://snap.stanford.edu/data/index.html (Accessed: March 16, 2023).